

Review



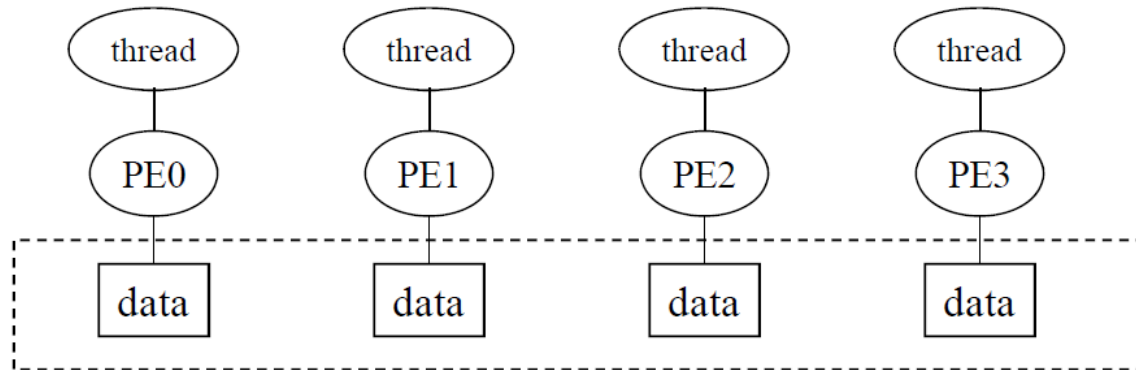
- Today's class:
 - Thread-level parallelism
 - Multicore/manycore processors

Flynn's Taxonomy of Computers



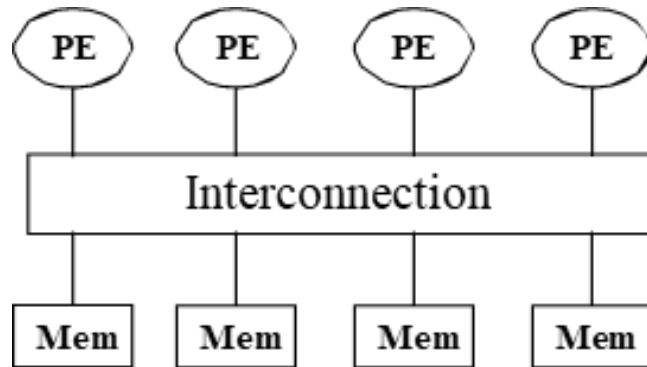
- Mike Flynn, “**Very High-Speed Computing Systems**,” Proc. of IEEE, 1966
- **SISD**: Single instruction operates on single data element
- **SIMD**: Single instruction operates on multiple data elements
 - Array processor
 - Vector processor
- **MISD**: Multiple instructions operate on single data element
 - Closest form: systolic array processor, data driven streaming processor
- **MIMD**: Multiple instructions operate on multiple data elements (multiple instruction streams)
 - Multiprocessor
 - Multithreaded processor

MIMD

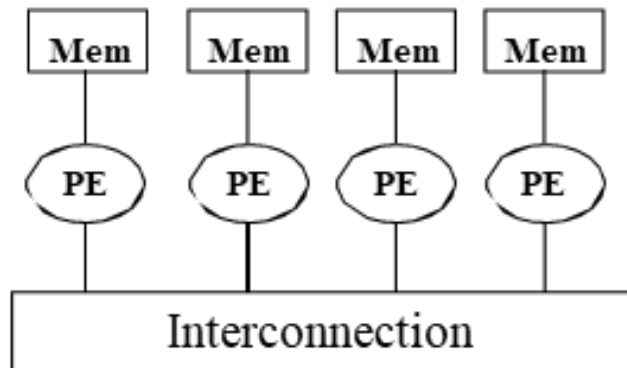


- ❑ Multiple threads executing on different data – However, if the threads are to cooperate to solve a problem (as opposed to solving different problems), there should be interaction between the programs and/or the data.
- ❑ Many flavors depending on the physical memory architecture and the virtual address space of each processing element (PE).
 - ❑ Address space = the range of memory addresses that the PE can access.

Physical Memory Architectures

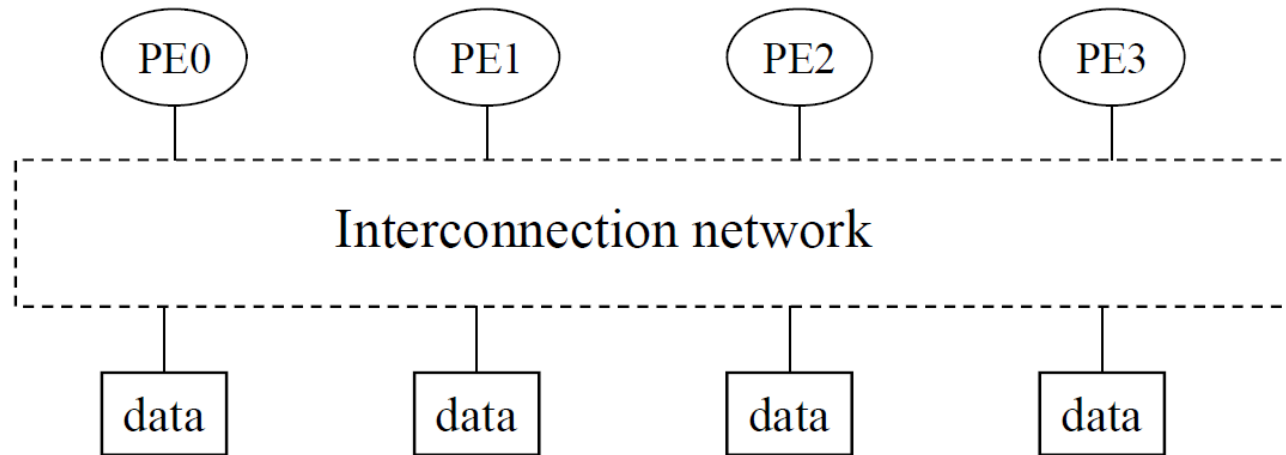


Global, shared memory (Symmetric Multi-Processors – SMP)



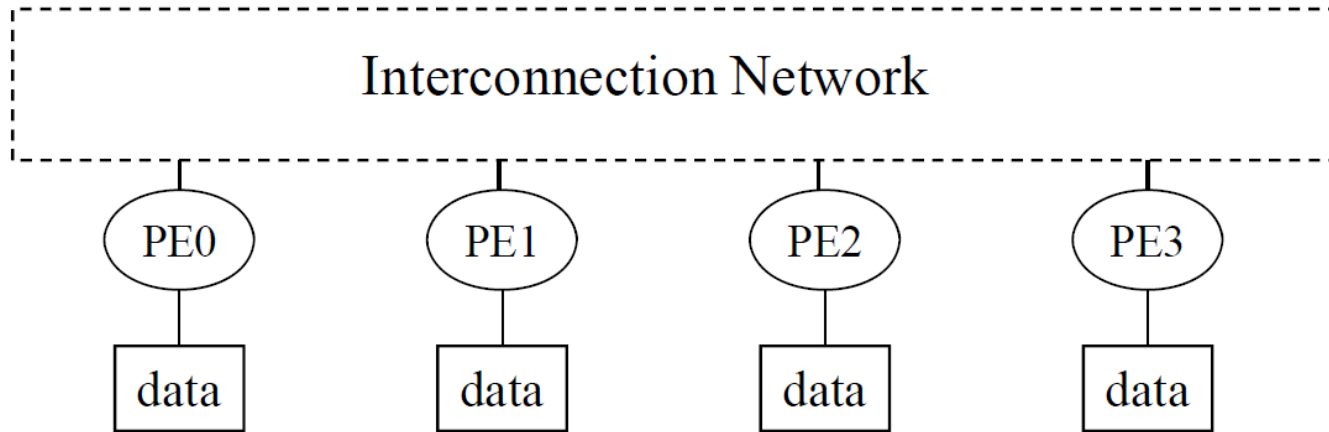
Distributed memory

Shared Address Space



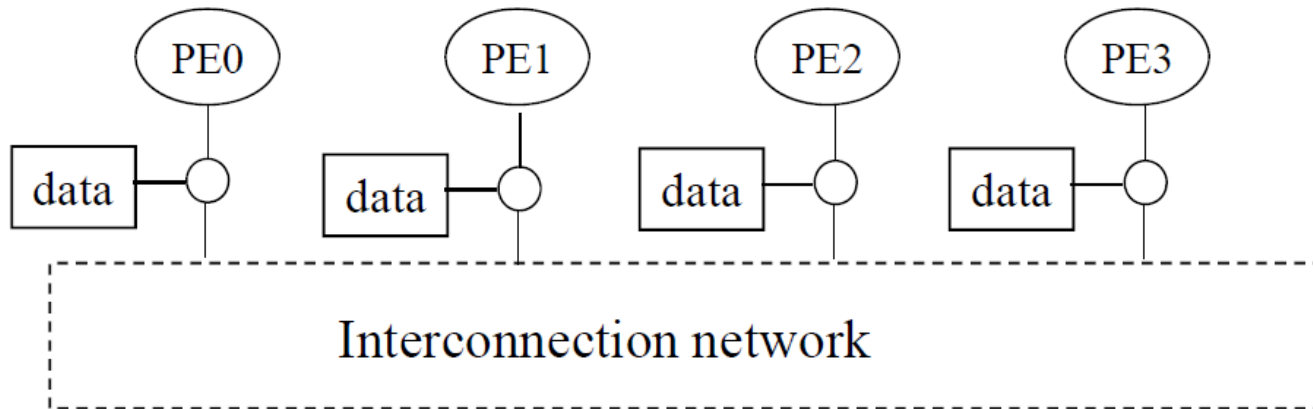
- ❑ Each of PE0, PE1, PE2 and PE3 can address (directly access) the same locations (shared virtual address space)
- ❑ No need for message passing – communicate through shared memory locations. E.g., OpenMP

Distributed Address Space



- ❑ Each of PE0, PE1, PE2 and PE3 **have separate** virtual address spaces.
- ❑ In order for PE_i to access data in the address space of PE_j, the two processes have to communicate through explicit messages. E.g. MPI

Distributed Shared Memory Systems



- ❑ Shared address space, but physically distributed memory.
 - ❑ No need for message passing – communicate through shared memory locations.
- ❑ Data is physically distributed, but a runtime system is responsible to access data that do not reside in the local memory.
- ❑ Results in the so called “Non Uniform Memory Access” – NUMA (as opposed to UMA, “Uniform Memory Access”)

Shared Memory multiProcessor (SMP)



❑ SMPs come in two styles

1. Uniform memory access (UMA) multiprocessors

Accesses to main memory take about the same amount of time no matter which processor requests the access and no matter which word is asked for

2. Nonuniform memory access (NUMA) multiprocessors

some memory accesses are much faster than other depending on which processor asks for which word

Programming NUMA is harder than programming UMA

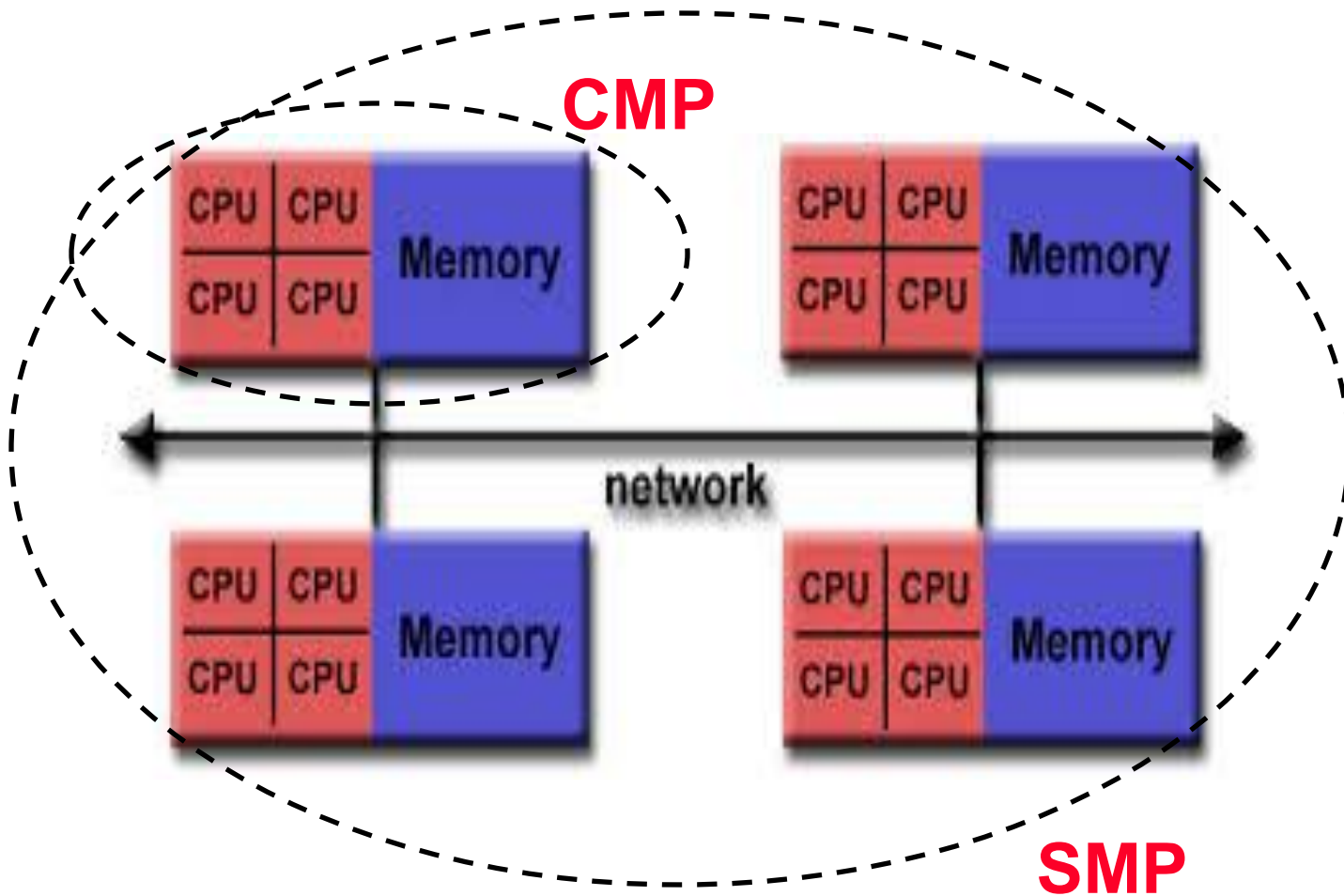
❑ Single physical address space shared by all cores

❑ Threads on cores coordinate/communicate through shared variables in memory (via loads and stores)

1. Use of shared data must be coordinated via **synchronization** primitives (locks) that allow access to data to only one core at a time (used to implement critical sections)

2. Caches must be kept **coherent** (read of a data item returns the most recently written value of that data item)

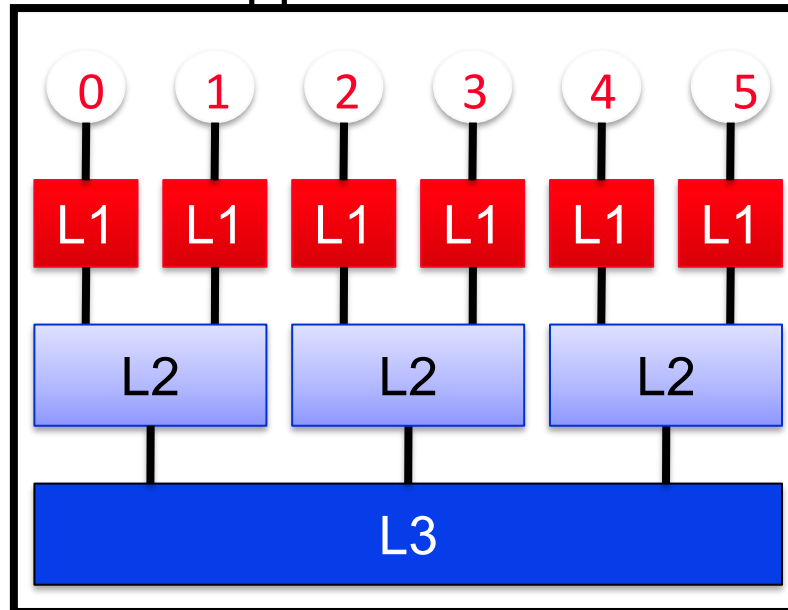
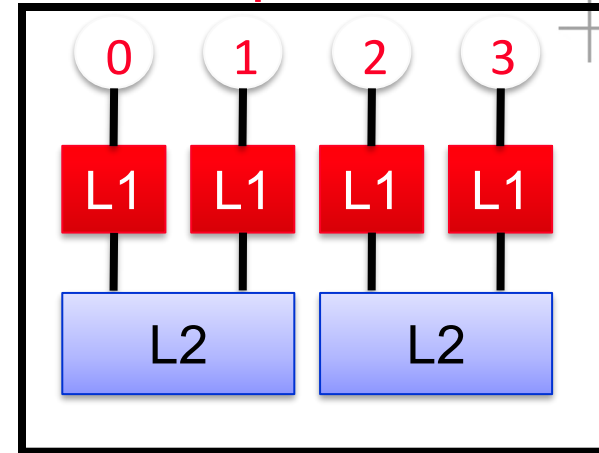
SMP (Shared Memory Multiprocessor) vs CMP (Multicore)



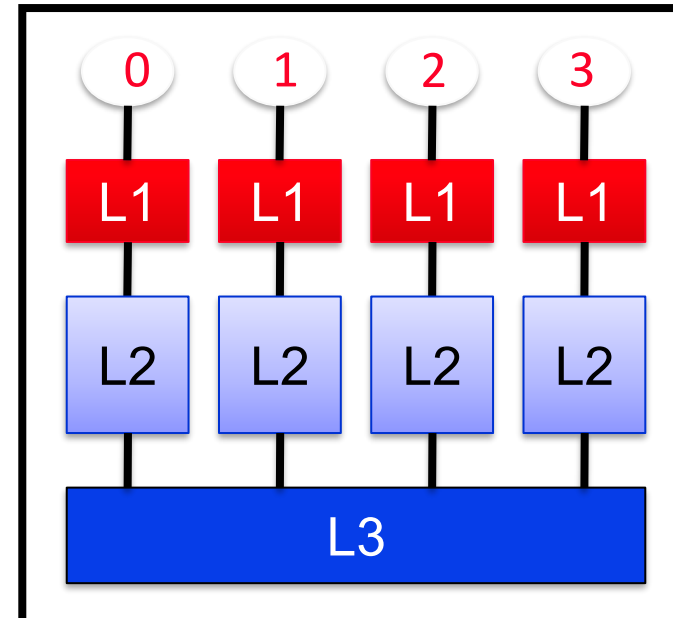
Multicore = Platform Variety

- ❑ Number/type of cores
- ❑ Levels and structure of the cache hierarchy
- ❑ Number of memory controllers (MCs) on-chip
- ❑ Type of on-chip interconnect
- ❑ GPU support?

Harpertown



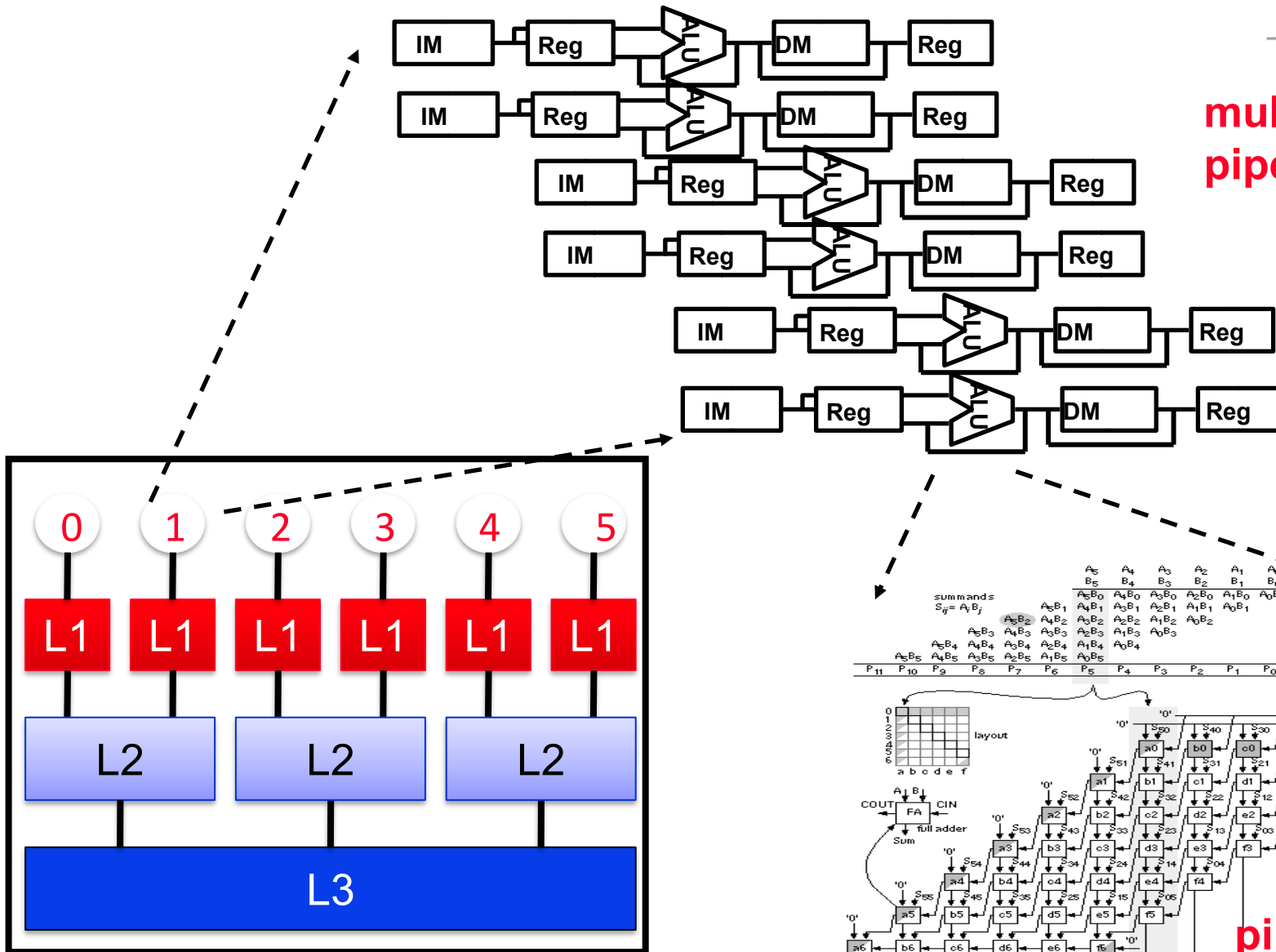
Dunnington



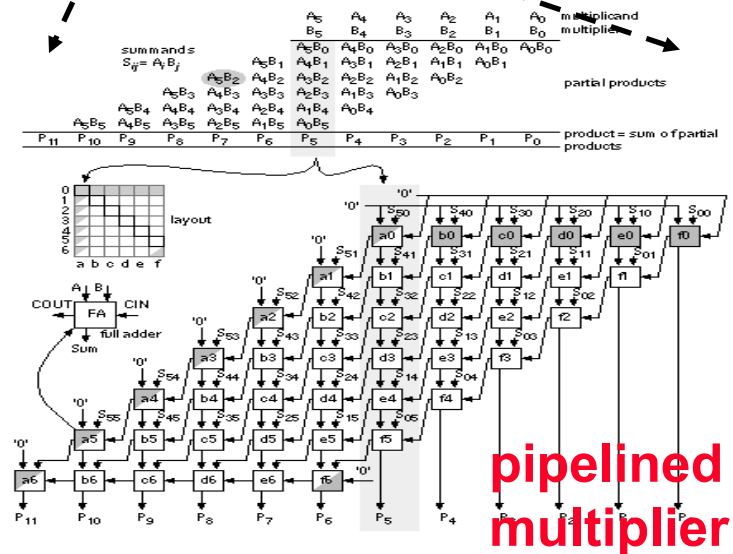
Nehalem

Hierarchical Parallelism

multi-issue
pipeline



CMP



pipelined
multiplier

The Trend



- ❑ Why not all cores on a single die?
 - ❑ Packaging difficulties
 - ❑ Scalability difficulties
 - ❑ Heating dissipation difficulties
 - ❑ Exceed reticle die area $\sim 800 \text{ mm}^2$ (technology challenge)
- ❑ Why not many processor chips
 - ❑ Communication overheads
 - ❑ NUMA overheads
 - ❑ Programming difficulties
- ❑ Chiplet Design
 - ❑ Pioneering Chiplet Technology and Design for the AMD EPYC™ and Ryzen™ Processor Families. AMD Inc. (ISCA 2021)

Shared Memory multiProcessor (SMP)



❑ SMPs come in two styles

1. Uniform memory access (UMA) multiprocessors

Accesses to main memory take about the same amount of time no matter which processor requests the access and no matter which word is asked for

2. Nonuniform memory access (NUMA) multiprocessors

some memory accesses are much faster than other depending on which processor asks for which word

Programming NUMA is harder than programming UMA

❑ Single physical address space shared by all cores

❑ Threads on cores coordinate/communicate through shared variables in memory (via loads and stores)

1. Use of shared data must be coordinated via **synchronization** primitives (locks) that allow access to data to only one core at a time (used to implement critical sections)

2. Caches must be kept **coherent** (read of a data item returns the most recently written value of that data item)

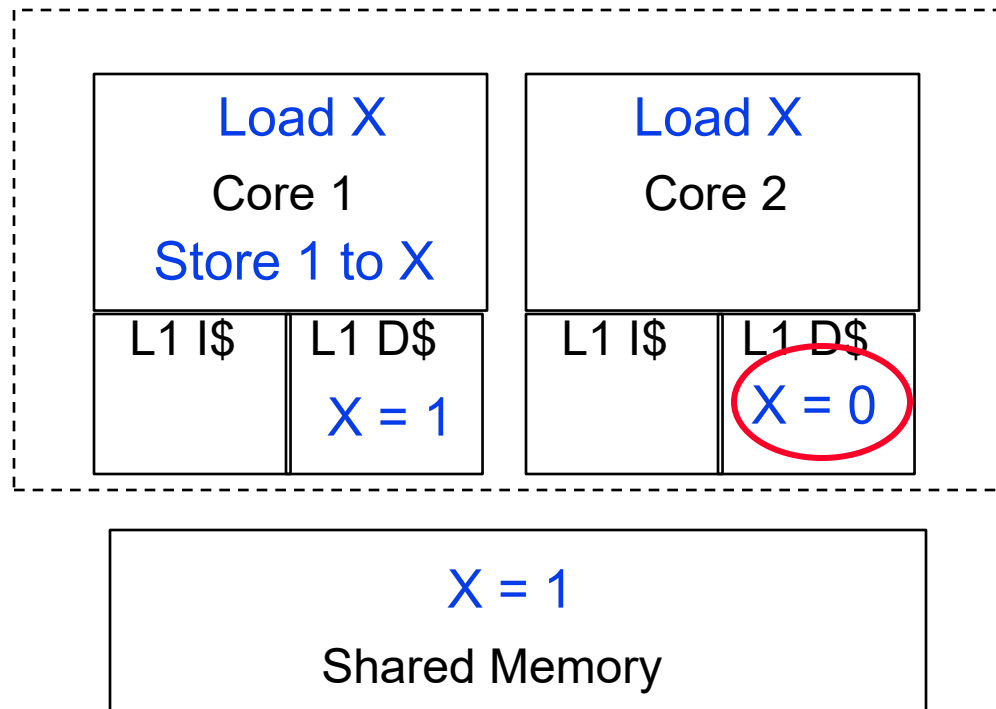
SMP Multithreaded Programming Model

- ❑ Creating multithreaded programs for multicores ?
- ❑ Programmer explicitly creates multiple software threads
 - ❑ Java has thread support built in, C/C++ supports P-threads library
 - ❑ Other tools such as OpenMP, Thread Blocks, Cilk Plus
 - ❑ Speedup via Thread-Level Parallelism (TLP)
- ❑ All loads & stores are to a single **shared memory** space
- ❑ A “thread switch” can occur at any time
 - ❑ SMT, FGMT
 - ❑ Pre-emptive multithreading by OS
 - Hardware timer interrupt occasionally triggers OS
 - Quickly swapping threads gives illusion of concurrent execution

Cache Coherence in Multicores

- ❑ Illustration of the **cache coherence** problem in a two core processor with private L1I\$ and L1D\$ caches and a (common) shared memory.

1. Core 1 does Load X
2. Core 2 does Load X
3. Core 1 does Store 1 to X
- ...
4. Core 1 does WB of X



A Coherent Memory System

- ❑ Any read of a data item should return the most recently written value of the data item
 - ❑ Coherence – defines **what values** can be returned by a read
 - Stores to the same location are **serialized** (two stores to the same location must be seen in the same order by all cores)
 - ❑ Consistency – determines **when** a written value will be returned by a read

Critical for performance

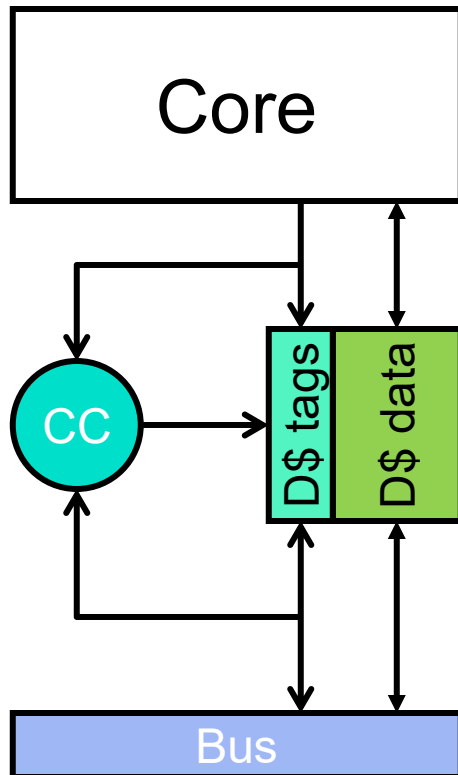
- ❑ To enforce coherence, caches must provide
 - ❑ Replication of shared data items in multiple cores' caches
 - Replication reduces both latency and contention for a read shared data item
 - ❑ Migration of shared data items to a core's local cache
 - Migration reduces the latency of the access the data and the bandwidth demand on the shared memory

Cache Coherence Protocols



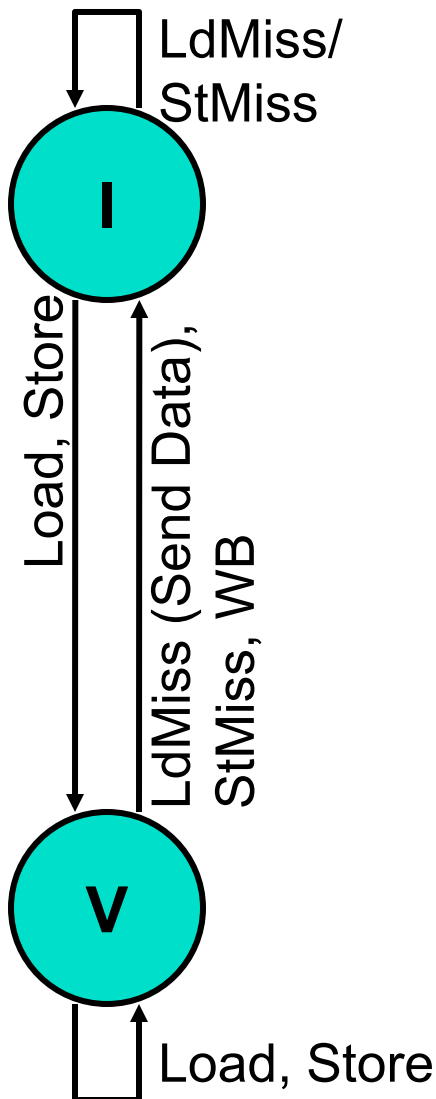
- ❑ Need a hardware mechanism to ensure cache coherence, the most popular of which is **snooping**
 - ❑ The cache controllers monitor (snoop) on the broadcast medium (e.g., bus) with duplicate address tag hardware (so they don't interfere with core's access to the cache) to determine if their cache has a copy of a block that is requested
- ❑ **Write invalidate protocol** – **writes** require exclusive access and **invalidate** all other existing copies
 - ❑ Exclusive access ensure that no other readable or writable copies of an item exists
- ❑ If two processors attempt to write the same data at the same time, one of them wins the race, causing the other core's copy to be invalidated. For the other core to complete, it must obtain the new data from the first core's cache – thus enforcing **write serialization**

Hardware Cache Coherence



- ❑ Write-back caches (rather than write-through) for performance reasons
- ❑ Coherence Controller (**CC**)
 - ❑ Monitors (“snoops”) bus traffic (addresses and data)
 - ❑ Executes the **coherence protocol**
 - What to do the with local copy when you see different things happening on bus
- ❑ Three core-initiated events
 - ❑ **Ld**: load **St**: store **WB**: write-back
- ❑ Two remote-initiated (bus) events
 - ❑ **LdMiss**: read miss from ***another*** core
 - ❑ **StMiss**: write miss from ***another*** core

VI (MI) coherence protocol



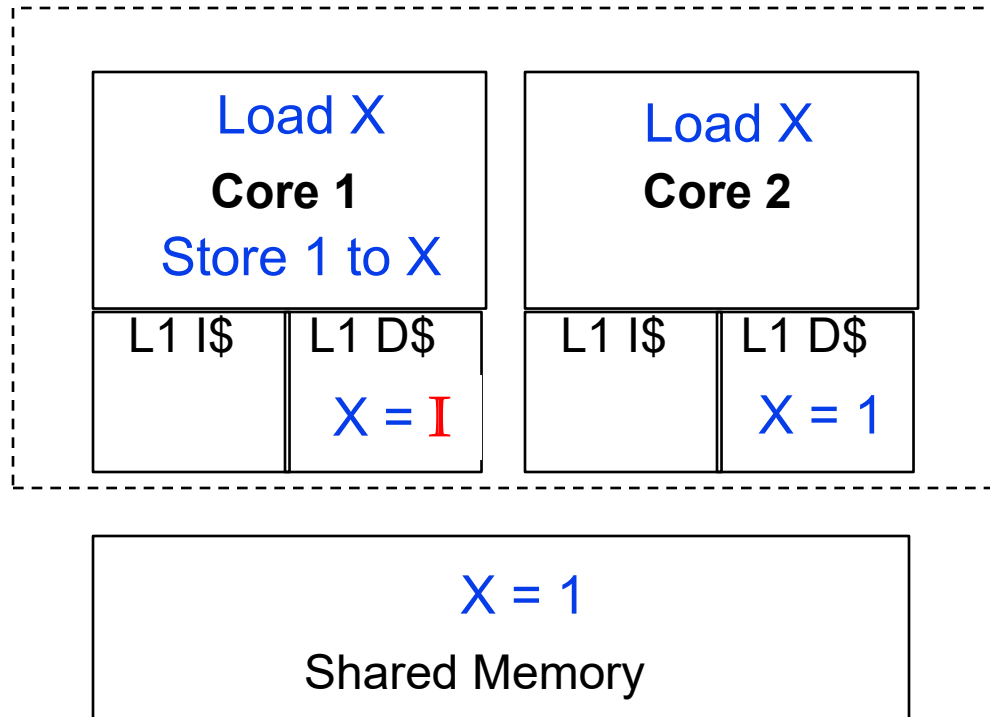
- ❑ Each cacheline has its own state machine
- ❑ VI (valid-invalid) protocol:
 - ❑ Two states (per block in cache)
 - V (valid): have block
 - I (invalid): don't have block
 - + Can implement with one bit (the valid bit)
- ❑ Protocol diagram (left)
 - ❑ If anyone **else** wants to read/write block
 - Give it up: transition to I state
 - ❑ WriteBack (WB) if your own copy is dirty
- ❑ This is an **write-invalidate** protocol
- ❑ **Write-update** protocol: copy data (write-through), don't invalidate
 - ❑ Sounds good, but wastes a lot of bus bandwidth

VI Protocol State Transition Table

State	<i>From This Core</i>		<i>From Other Cores</i>	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Miss → V	Miss → V	---	---
Valid (V)	Hit	Hit	Send Data → I	→ I

- ❑ Rows are “states” (**I** vs **V**) of data blocks in the caches
- ❑ Columns are “events” in the cores
 - ❑ WriteBack events not shown
 - ❑ **V** → **I** on a LdMiss also updates memory (Send Data) **if** block is Dirty
- ❑ Memory sends data when no core responds

Example of VI Snooping Protocol



LdMiss seen by Core 1, so Core 1 invalidates its copy

StMiss seen by Core 2, so Core 2 invalidates its copy

LdMiss seen by Core 1, so Core 1 does a Send Data and then invalidates its copy

- When the second miss by Core 2 occurs, Core 1 responds with the data value for Core 2 (canceling the response from the shared memory), updating the value of X in the shared memory, and invalidating its own copy

VI → MSI

❑ VI protocol is inefficient

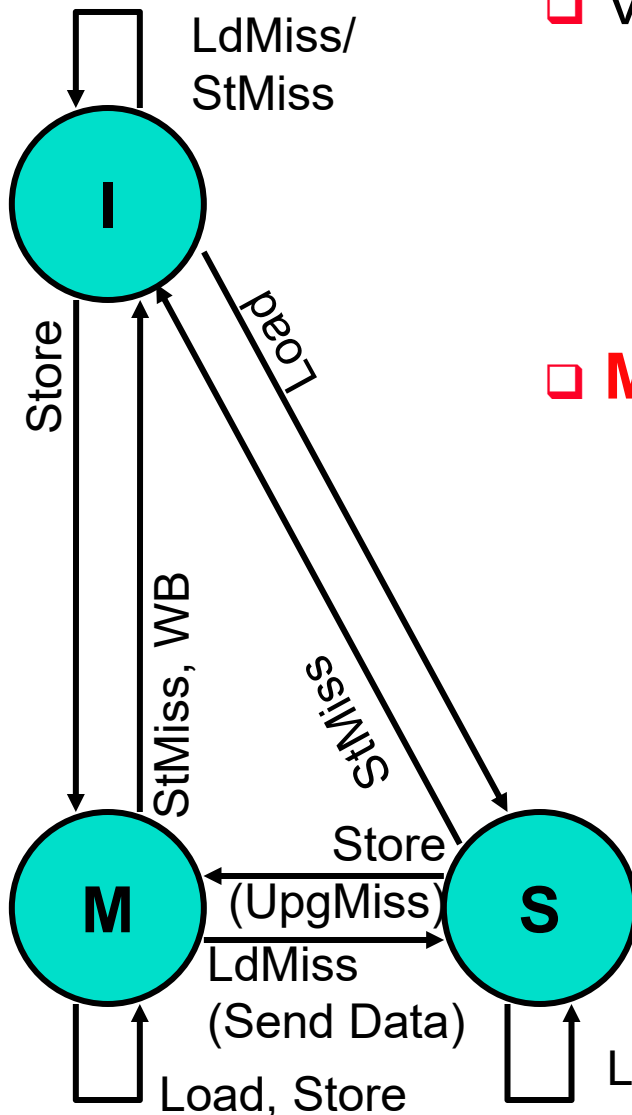
- Only **one** cached copy allowed in entire system
- Multiple caches copies can't exist even if they **are** all **read-only**

❑ MSI (modified-shared-invalid)

- ❑ Fixes problem: splits “V” state into two states
 - **M (modified)**: local dirty copy
 - **S (shared)**: local clean copy
 - **I (invalid)**
 - Takes 2 bits to implement

❑ Allows **either**

- Multiple read-only copies (**S**-state) **--OR--**
- **Single** read/write copy (**M**-state)

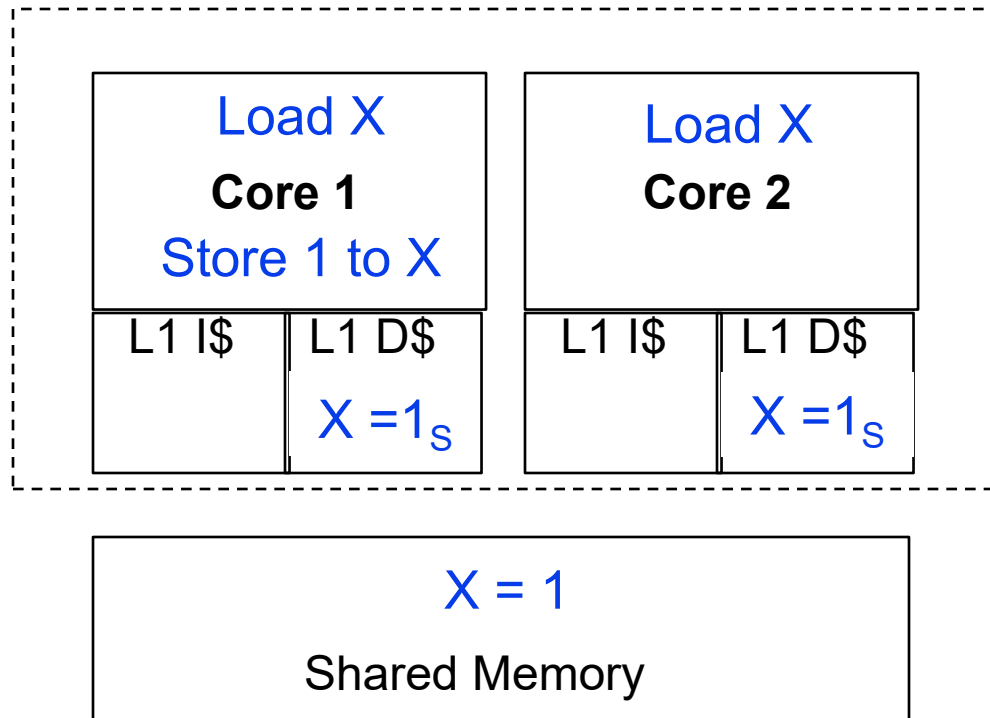


MSI protocol state transition table

State	<i>From This Core</i>		<i>From Other Cores</i>	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Miss → S	Miss → M	---	---
Shared (S)	Hit	Upg Miss → M	---	→ I
Modified (M)	Hit	Hit	Send Data → S	Send Data → I

- ❑ M → S transition on a LdMiss also updates memory (Sends Data) since the block is Dirty

Example of MSI Snooping Protocol



LdMiss seen by Core 1 (no action required)

Core 1 marks X as modified and sends out an UpgMiss

StMiss (UpgMiss) seen by Core 2 so Core 2 invalidates its copy

LdMiss seen by Core 1, so Core 1 does a Send Data of X and marks its copy as shared

- ❑ When the second miss by Core 2 occurs, Core 1 responds with the value canceling the response from the shared memory (and also updating the value of X in the shared memory and marking its own copy as shared)

Other Coherence Protocols



- ❑ Another write-invalidate protocol used in the Pentium 4 (and many other processors) is **MESI** with four states:
 - ❑ **M**odified – same (local dirty copy)
 - ❑ **E**xclusive – only one copy of the shared data is allowed to be cached (local clean copy)
 - Since there is only one copy of the data, write hits don't need to send UpgMiss broadcast
 - ❑ **S**hared – same (multiple copies of the shared data may be cached (i.e., data permitted to be cached with more than one core))
 - ❑ **I**nvalid – load miss is E if no other core is caching the data, else its S
- ❑ Other variations include MOSI, MOESI, MERSI, MESIF, Berkeley, Firefly and Dragon
- ❑ Snooping protocols (and buses) don't scale well to many-cores
 - ❑ Alternative is directory based, non-broadcast coherence protocols where a directory in memory keeps track of data ownership

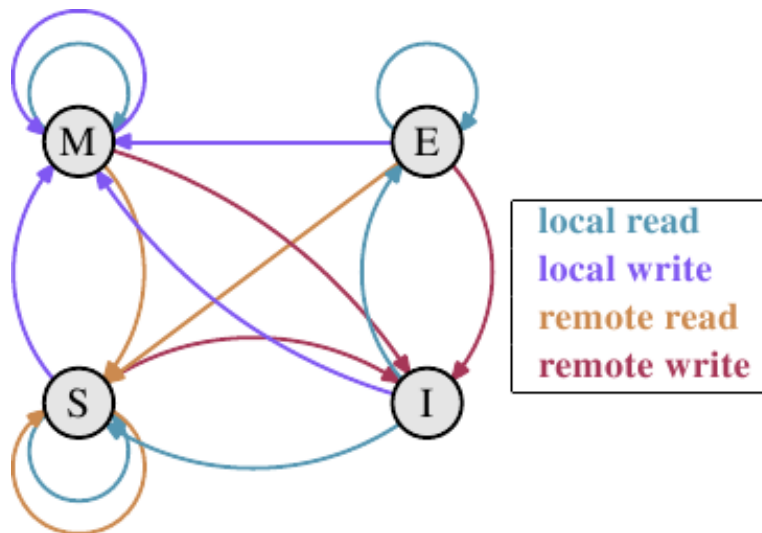
❑ Last Class:

- ❑ Multicore/manycore processors
- ❑ Snooping-based cache coherence

❑ Today's class:

- ❑ Directory-based cache coherence

The MESI protocol



- ❑ On a read miss (local block is invalid), load the block and change its state to
 - ❑ “exclusive” if it was uncached in memory
 - ❑ “shared” if it was already shared, modified or exclusive
 - if it was modified, the owner will send you a clean copy
 - if it was modified or exclusive, the previous owner will change the state of the block to “shared” in its cache.
- ❑ On a write miss: same as read miss, except set the state to “modified”
 - ❑ copies in other caches (if any) are invalidated
- ❑ On a write hit to a “modified” block, do nothing
- ❑ On a write hit to an “exclusive” block change the block to “modified”
 - ❑ no need for invalidation.
- ❑ On a write hit to a “shared” block change the block to “modified” and invalidate the other cached copies.
- ❑ When a modified block is evicted, write it back.

The MESI protocol



- ❑ If MESI is implemented as a snooping protocol, then the main advantage over the three state protocol is when a read to an uncached block is followed by a write to that block.
 - ❑ After the uncached block is read, it is marked “exclusive” – (need a scheme to know that it was uncached).
 - ❑ Note that, when writing to a shared block, the transaction has to be posted on the bus so that other sharers invalidate their copies.
 - ❑ But when writing to an exclusive block, there is no need to post the transaction on the bus. Hence, by distinguishing between shared and exclusive states, we can avoid bus transactions when writing on an exclusive block.
 - ❑ However, now a cache that has an “exclusive” block has to monitor the bus for any read to that block. Such a read will change the state to “shared”.
 - ❑ What if a block was shared in two caches and is evicted from one of them. Should we detect this case and set the block to Exclusive in the other cache?

Directory-based Coherence (§ 5.4)

- ❑ Idea: Implement a “directory” that keeps track of where each copy of a block is cached and its state in each cache (note that with snooping, the state of a block was kept only in the cache).
- ❑ Processors must consult the directory before caching blocks from memory. If block is “modified”, then its “owner” should provide the most up-to-date copy.
- ❑ When a block in memory is updated (written), the directory is consulted to either update or invalidate other cached copies.
- ❑ Eliminates the overhead of broadcasting/snooping (bus bandwidth) – Hence, scales up with the numbers of processors that would saturate a single bus.
- ❑ latency??

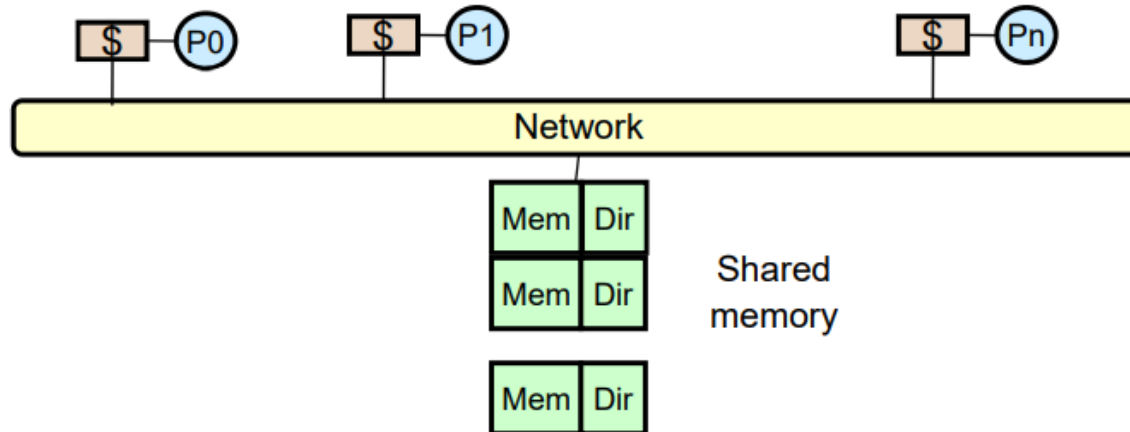
Directory-Based Coherence



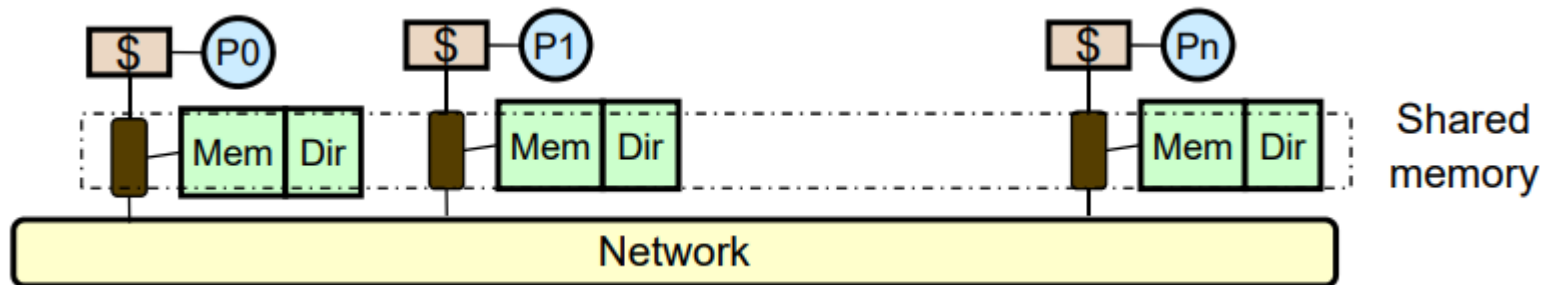
- ❑ No centralized bus and no broadcast for coherence traffic!
- ❑ Each cache/memory contains a portion of the directory
- ❑ Directory entry tells
 - ❑ Whether the block is in the cache
 - ❑ If so, where (which cache) and its status
- ❑ **Algorithm:**
 - ❑ Use physical address to go to the directory
 - ❑ Check whether the block is in the cache
 - ❑ If yes:
 - Go to the cache that contains the block
 - ❑ If no:
 - Issue a memory request {we have a cache miss}
 - The block is brought to the requesting core/cache
 - The directory is updated
- ❑ There are many ways to build a directory (e.g., distributed vs shared)

Directory-based Coherence

- ❑ The memory and the directory can be centralized



- ❑ Or distributed



- ❑ Alternatively, the memory may be distributed but the directory can be centralized.
- ❑ Or the memory may be centralized but the directory can be distributed.

Directory-based Coherence (§ 5.4)

- ❑ The location (home) of each memory block is determined by its address.
- ❑ A controller decides if access is Local or Remote
- ❑ As in snooping caches, the state of every block in every cache is tracked in that cache (modified/dirty, shared/clean, invalid).
- ❑ In addition, a directory entry keeps track of where the block is cached. Accordingly, a block can be in one of the following states:
 - ❑ Uncached: no processor has it (not valid in any cache)
 - ❑ Shared/clean: cached in one or more processors and memory is up-to-date
 - ❑ modified/dirty: one processor (owner) has data; memory out-of-date

Enforcing coherence

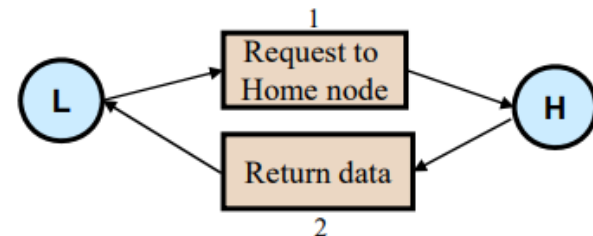


- ❑ Coherence is enforced by exchanging messages between nodes/PEs/cores.
- ❑ Three types of nodes may be involved
 - ❑ **Local requester node (L)**: the node that reads or write the cache block
 - ❑ **Home node (H)**: the node that stores the block (and its directory entry) in its memory -- may be the same as L
 - ❑ **Remote nodes (R)**: other nodes that have a cached copy of the requested block.
- ❑ When L encounters a **Read Hit**, it just reads the data
- ❑ When L encounters a **Read Miss**, it sends a message to the home node, H, of the requested block – three cases may arise:
 - ❑ The directory indicates that the block is “not cached”
 - ❑ The directory indicates that the block is “shared/clean”
 - ❑ The directory indicates that the block is “modified”

What happens on a read miss? (when block is invalid in local cache)

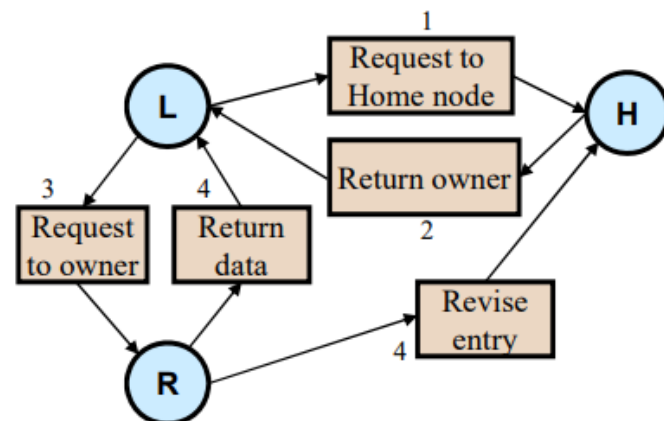
❑ Read miss (if block is shared or uncached)

- ❑ L sends request to H
- ❑ H sends the block to L
- ❑ state of block is “shared” in directory
- ❑ state of block is “shared” in L



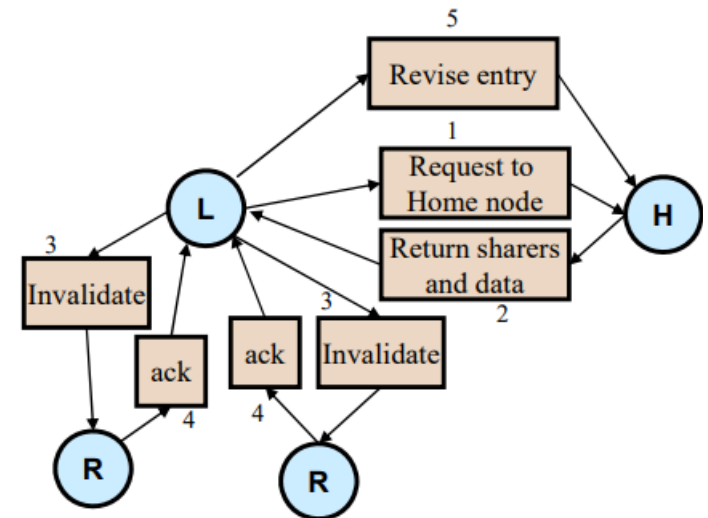
❑ Read miss (if block is modified in another cache)

- ❑ L sends request to H
- ❑ H informs L about the block owner, R
- ❑ L requests the block from R
- ❑ R send the block to L
- ❑ L and R set the state of block to “shared”
- ❑ R informs H that it should change the state of the block to “shared”



What happens on a write miss? (when block is invalid in local cache)

- ❑ Write miss to an uncached block
 - ❑ similar to a read miss to an uncached block except that the state of the block is set to “modified”
- ❑ Write miss to a block that is modified in another cache
 - ❑ similar to a read miss to an modified block except that the state of the block is set to “modified” in H and L and to “Invalid” in R.
- ❑ Write miss to a shared block
 - ❑ L sends request to H
 - ❑ H sets the state to “modified”
 - ❑ H sends the block to L
 - ❑ H sends to L the list of other sharers
 - ❑ L sends invalidating messages to each sharers (R)
 - ❑ Each R sets block’s state to “invalid” and send “ack” to L
 - ❑ L sets the block’s state to “modified” and notifies H



What happens on a write hit? (when block is shared or modified in local cache)

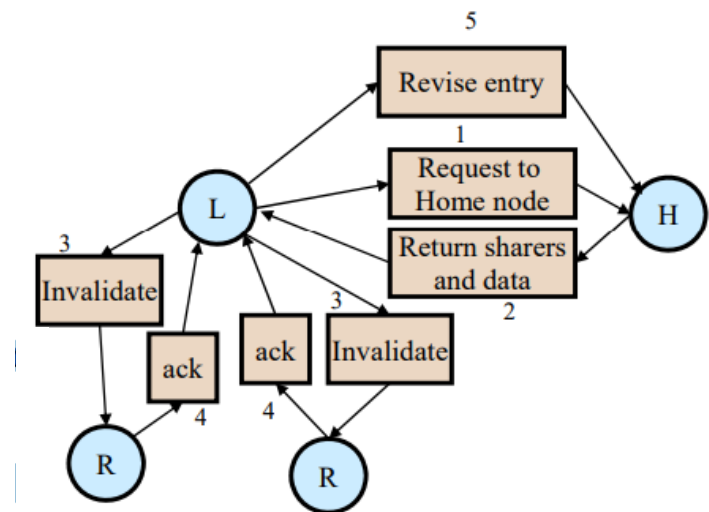
❑ If the block is “modified” in L, just write the data

❑ If the block is “shared” in L

- ❑ L sends a request to H to have the block as “modified”
- ❑ H sets the state to “modified”
- ❑ H informs L of the block’s other sharers
- ❑ L sets the block’s state to “modified”
- ❑ L sends invalidating messages to each sharers (R)
- ❑ R sets block’s state to “invalid”

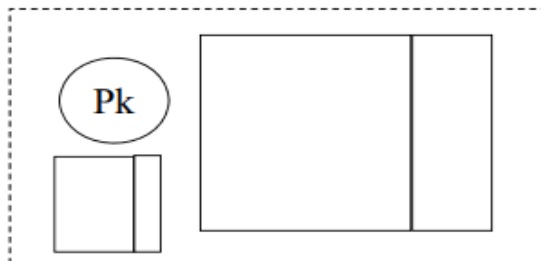
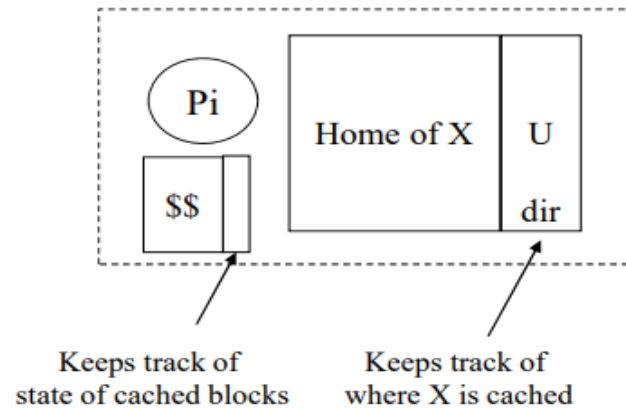
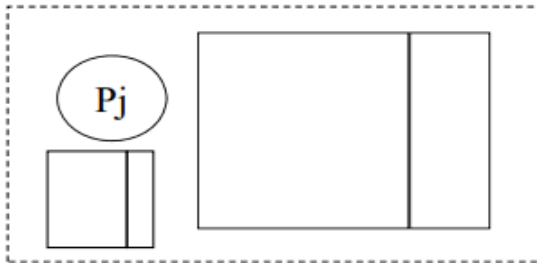
❑ Additional complexity:

- ❑ We need a “busy” state to handle simultaneous requests to the same block. For example, if there are two writes to the same block – it has to be serialized. Reason: order of events depends on message orders, which is non-deterministic.
- ❑ Book to read: A Primer on Memory Consistency and Cache Coherence, Second Edition. Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, David A. Wood February 2020



MSI Directory-based coherence - example

- ❑ Three cores P_i , P_j , P_k
- ❑ P_i is the home node of X (i.e., the memory address of X is in the memory partition on node P_i)
- ❑ X is un-cached (U) in the home directory
- ❑ P_j reads X

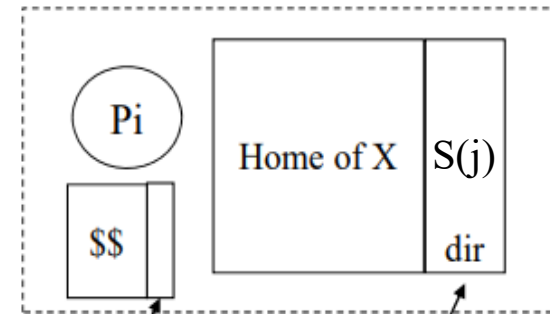
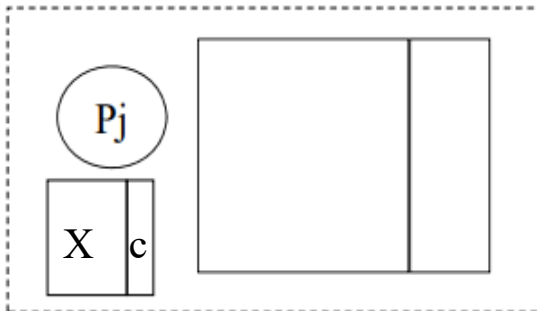


Possible scenario:

- P_j reads X

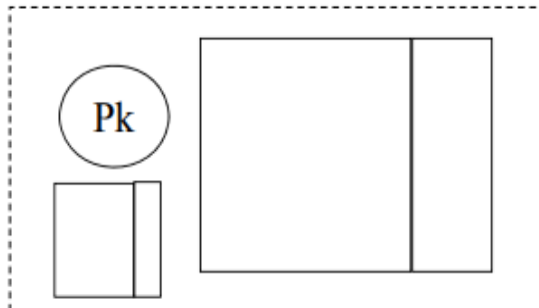
MSI Directory-based coherence - example

- ❑ Directory holds X is shared by Pj
- ❑ Pj has the copy of X and it is a clean copy.



Keeps track of
state of cached blocks

Keeps track of
where X is cached

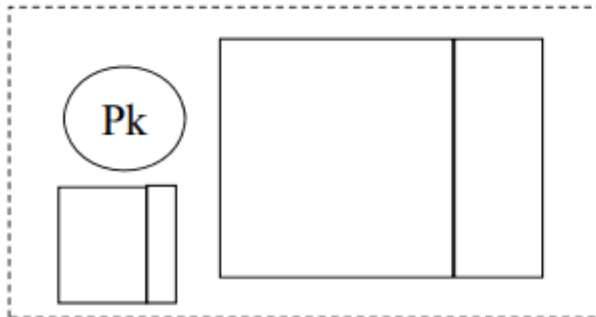
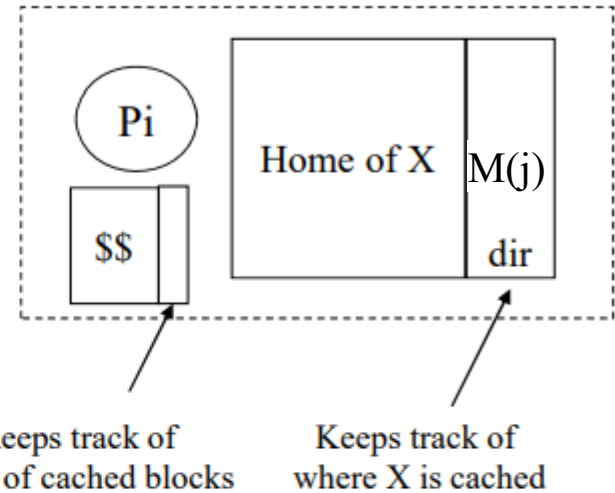
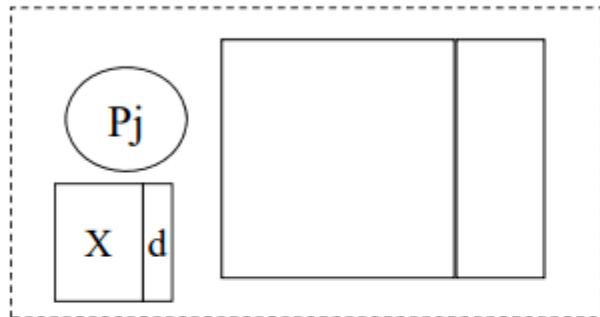


Possible scenario:

- Then Pj writes to X

MSI Directory-based coherence - example

- ❑ Directory holds X is modified by Pj
- ❑ Pj has the copy of X and it is a dirty copy.

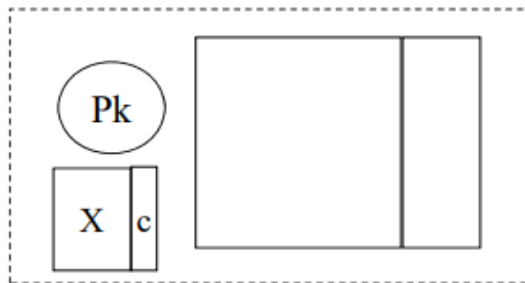
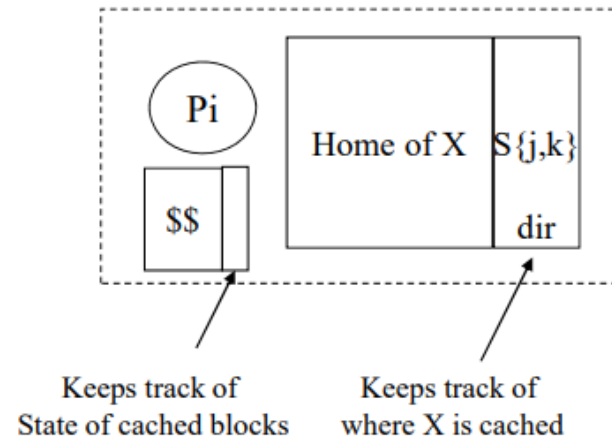
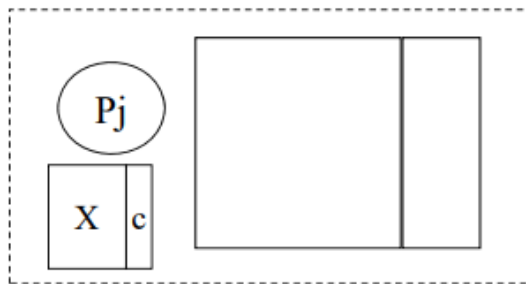


Trace the state of X if:

- P_k reads X

MSI Directory-based coherence - example

- ❑ Directory holds X is shared by P_j and P_k
- ❑ P_j and P_k has the clean copy of X.

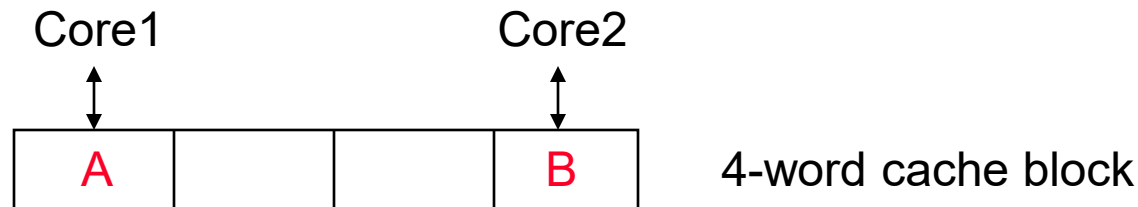


Coherence Misses

1. **True-sharing misses** arise from communication of data through cache-coherence mechanism
 - ❑ Invalidates due to write to shared word
 - ❑ Reads by another core of modified block in different cache
 - ❑ This miss would still occur if block size were 1 word
2. **False-sharing misses** when a block is invalidated because some word in the block, other than the one being read, is written into
 - ❑ Invalidation does not cause a new value to be communicated, but only causes an extra cache miss
 - ❑ Fundamental reason: coherence protocol is applied at cache line/block granularity!
 - ❑ Block is shared, but no word in block is actually shared
 - ❑ This miss would not occur if block size were 1 word

Block Size Effects

- ❑ Writes to one word in a multi-word block mean that the full block is invalidated
- ❑ Multi-word blocks can also result in **false sharing**: when two cores are writing to two different variables that happen to fall in the same cache block
 - ❑ False sharing increases cache miss rates



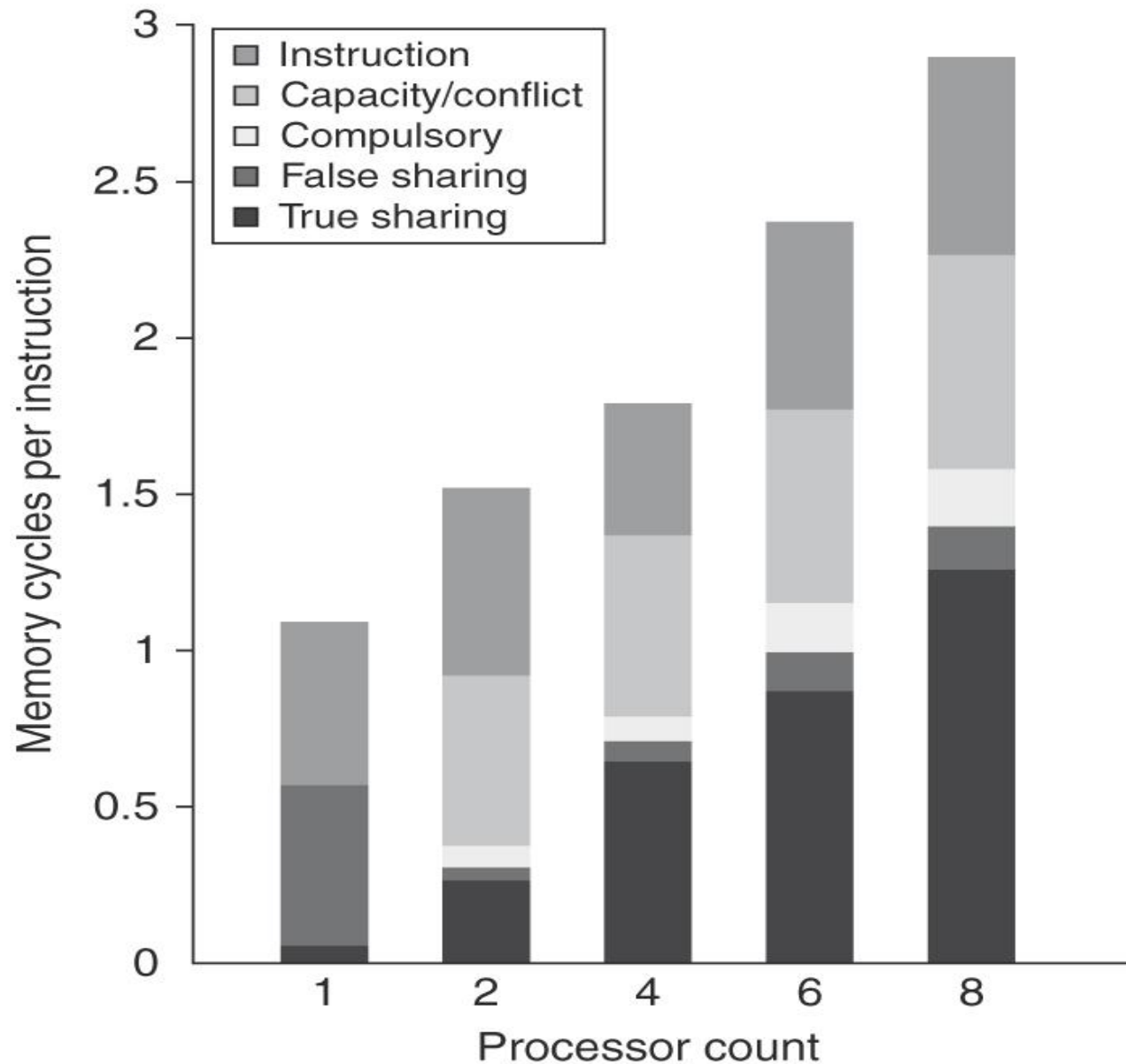
- ❑ Compilers can help reduce false sharing by allocating highly correlated data to the same cache block
 - ❑ How to know?
 - ❑ Padding? Good or bad?

Example: True Sharing vs False Sharing

Assume d1 and d2 in same cache block.
P1 and P2 both read d1 and d2 before.

Step	P1	P2	True, False, Hit? Why?
1	Write d1		Invalidate the cache block in P2
2		Read d2	False miss; d1 irrelevant to P2
3	Write d1		Invalidate the cache block in P2
4		Write d2	Invalidate the cache block in P1
5	Read d2		True miss; they are sharing d2

Memory Cycle Distribution



Cache Coherence Summary

- ❑ Problematic when having multi-core and multiple private cache
- ❑ Coherence protocols vs coherence implementation
 - ❑ VI, MSI, MESI, etc.
 - ❑ Snooping, directory based.
- ❑ Coherence protocol applies on each cacheline
- ❑ False sharing degrades performance (not correctness) due to coherent cache design